

Especificación Técnica para Implementar un Sistema Seguro de Autenticación y Autorización Basado en JWT y Roles

Esta documentación establece los requisitos funcionales y técnicos que toda solución debe cumplir para resolver de manera profesional el desafío propuesto en la lección práctica sobre autenticación y autorización basada en tokens JWT. El objetivo no es solo funcional, sino arquitectónico y de seguridad, en línea con estándares de desarrollo backend contemporáneo.

1. Modelo de Seguridad

1.1. Hashing de Contraseñas

- Toda contraseña debe ser hasheada usando `bcrypt`, con un mínimo de 10 salt rounds.
- El almacenamiento de contraseñas en texto plano, incluso durante el proceso de prueba, está estrictamente prohibido.
- El algoritmo debe ser resistente a ataques por diccionario, fuerza bruta y ataques de precomputación.

1.2. Generación y Firma del JWT

- La autenticación exitosa debe emitir un JWT firmado con una clave secreta segura o clave privada (si se utiliza RS256).
- El JWT debe incluir, al menos:
 - `userId`
 - `username`
 - `role`
 - `iat` (issued at)
 - `exp` (expiration)

1.3. Expiración del Token

- El token debe tener una validez máxima de 15 a 60 minutos.
- Se recomienda implementar un sistema de *refresh token* seguro para prolongar sesiones sin comprometer la seguridad.

2. Roles y Permisos

2.1. Definición de Roles

- **Administrador**: Acceso total al sistema.
- **Editor**: Puede crear y modificar entidades asignadas, pero no eliminarlas.
- **Usuario Común**: Acceso exclusivamente de lectura a recursos asignados.

2.2. Encapsulamiento de Permisos

- La lógica de permisos no debe estar en el frontend ni dispersa en los controladores.
- Debe centralizarse en un middleware o una función de autorización reutilizable.

3. Gestión de Sesiones y Acceso

3.1. Middleware de Autenticación

- Verifica la validez, firma y expiración del JWT en cada solicitud protegida.
- Rechaza la solicitud con un código **401 Unauthorized** si el token no es válido o ha expirado.

3.2. Middleware de Autorización

- Verifica el rol extraído del JWT y determina el acceso a la ruta o recurso solicitado.
- Retorna código **403 Forbidden** cuando un rol carece de privilegios suficientes.

3.3. Cierre de Sesión

- Aunque el backend sea *stateless*, se requiere revocación segura:
 - Eliminación del token en el lado cliente (`localStorage`, `sessionStorage`, o cookies).
 - En sistemas avanzados: implementar listas de revocación en servidor o bases temporales de tokens expirados/revocados.

4. Arquitectura de Rutas y Recursos

4.1. Segmentación de Endpoints

- `/auth/register`: Registro de nuevos usuarios. Por defecto, asigna el rol `Usuario Común`.
- `/auth/login`: Genera un JWT válido al autenticarse con credenciales correctas.
- `/auth/logout`: Invalida el JWT en el cliente.
- `/projects`, `/tasks`: Rutas protegidas, con acceso condicionado por el rol del usuario autenticado.

4.2. Protección Total del Backend

- No debe existir ninguna ruta que devuelva datos sensibles sin validación de token.
- Las rutas de administración (creación, edición, eliminación) deben ser accesibles solo a roles adecuados.

5. Persistencia de Datos

5.1. Estructura de Usuario

- Campos mínimos: `id`, `username`, `password_hash`, `role`, `created_at`.
- Se recomienda evitar almacenar roles como texto plano y en su lugar utilizar una tabla relacional o constantes tipadas.

5.2. Relación con Proyectos/Tareas

- Cada entidad debe tener campos de control que permitan filtrar por propietario y rol asignado.
- El backend debe verificar que un usuario solo acceda a recursos que le corresponden.

6. Validaciones y Seguridad Adicional

6.1. Sanitización de Entradas

- Todas las entradas deben pasar por validadores que eviten inyecciones SQL, XSS y otras formas de input malicioso.

6.2. Auditoría de Accesos

- Se recomienda registrar las acciones realizadas por cada usuario en un log de auditoría con timestamp, endpoint, método HTTP, y resultado.

6.3. Extensibilidad

- El diseño debe permitir fácilmente añadir roles, funcionalidades avanzadas (como MFA) y otras entidades sin romper el sistema actual.

7. Pruebas y Entrega

7.1. Casos de Prueba Obligatorios

- Usuario común intentando borrar un recurso → debe fallar.
- Editor intentando editar un recurso no asignado → debe fallar.
- Administrador accediendo a todo el sistema → debe funcionar.
- Token manipulado → debe fallar con 401.

7.2. Evidencias

- Código fuente con rutas protegidas y roles implementados.
- Capturas de pantalla del flujo de autenticación, verificación de roles y restricciones.
- Informe detallado explicando el flujo de autenticación, cómo se gestionan los tokens y cómo se aplica la autorización.

